Underscorify Substring

AlgoExpert · Hard · Strings

The problem

- Wrap every occurrence of `sub` in `str` with underscores, merging overlapping or adjacent occurrences into one wrapped block.

Intuition

- Find all occurrences, merge overlapping or touching ones, then insert underscores.

Brute force vs optimal

- Insert underscores per occurrence then merge → messy
- Mark covered chars, then emit runs → O(n · m)
- Or merge intervals

Approach

- 1. Collect occurrence intervals
- 2. Merge overlaps and adjacency
- 3. Insert underscores at boundaries

Worked example

```
testthis is a testtest to see if testestest it works
```

Find every occurrence of 'test'

Worked example

```
^^^^          ^^^^^^^^          ^^^^^^^^^^
```

```
testthis is a testtest to see if testestest it works
```

Mark covered characters

Worked example

```
_test_this is a _testtest_ to see if _testestest_ it works
```

Underscore each marked run

Java solution

```java
public class UnderscorifySubstring {
    public static String underscorifySubstring(String str, String sub) {
        boolean[] mark = new boolean[str.length()];
        for (int i = str.indexOf(sub); i != -1; i = str.indexOf(sub, i + 1))
            for (int j = i; j < i + sub.length(); j++) mark[j] = true;
        StringBuilder sb = new StringBuilder();
        // adjacent occurrences (end of one == start of next) also merge, which marking handles naturally
        for (int i = 0; i < str.length(); i++) {
            if (mark[i] && (i == 0 || !mark[i - 1])) sb.append('_');
            sb.append(str.charAt(i));
            if (mark[i] && (i == str.length() - 1 || !mark[i + 1])) sb.append('_');
        }
        return sb.toString();
    }
}
```

Complexity

- Time: O(n*m)
- Space: O(n)

Edge cases & pitfalls

- Overlapping occurrences
- Adjacent occurrences merge
- No occurrences → unchanged

Interview tips & follow-ups

- Mention the interval-merge approach

Recap

- Find all occurrences, merge overlapping or touching ones, then insert underscores.
- Complexity: O(n*m)